Belem DeFi Risk — Cadrage & Guideline

Oct 10, 2026 · @Max

TL;DR

Projet recommandé : « StackTrace » (nom provisoire), un rayon X pour position DeFi. On donne une position réelle (ex. PT-sUSDe de Pendle posé en collatéral sur Morpho), l'outil reconstruit tout seul son graphe de dépendances depuis la chaîne, on déclenche un incident sur n'importe quel nœud, la perte se propage jusqu'à la position, et l'outil rend un verdict **STAY / PARTIAL / EXIT** chiffré et expliqué.

Trois choses nous différencient des autres équipes :

1. **Chaque dépendance est prouvée** : adresse de contrat, appel on-chain ou source API affichés à côté de chaque arête. Zéro graphe dessiné à la main présenté comme automatique.
2. **Le verdict donne un seuil, pas juste un score** : « sortez si vous estimez la probabilité que l'incident se matérialise au-dessus de 18 % ». C'est exactement l'arbitrage 11 % de slippage du brief, rendu explicite.
3. **On le prouve sur un vrai incident** : replay d'une crise réelle (cascade Stream xUSD → Elixir deUSD → marchés de prêt, novembre 2025) pour montrer que l'outil aurait trouvé le chemin de contagion.

Périmètre volontairement étroit : **Ethereum mainnet, 2 positions types proches du book Belem**, une 3e en bonus. Rien n'est codé tant que les 3 questions de la section *Test* ne sont pas validées entre vous et, idéalement, avec un mentor Belem.

1. Empathize : pour qui on construit

Le jury, c'est l'équipe risk de Belem, et elle a déjà l'alerte. Ce qui lui manque, c'est l'étape d'après : « qu'est-ce que ça fait à mon book, combien, et je fais quoi ». Le brief le dit mot pour mot : le dur n'est pas de remarquer l'incident.

Ce qu'on sait d'eux (brief) : ~400 M$ gérés, stratégies market-neutral de yield sur stablecoins, ETH, BTC, SOL ; 39+ protocoles et 20+ chaînes suivis ; zéro incident, zéro défaut, zéro liquidation depuis 2020 ; un scorecard de pré-sélection et du monitoring temps réel déjà en place. Leur obsession affichée à l'oral : les dépendances.

Persona : Léa, risk manager d'astreinte, 3 h du matin

Une alerte tombe : « l'oracle de sUSDe dévie » ou « exploit sur le protocole X ». Ses questions, dans l'ordre :

1. Est-ce que je suis exposée, et par quel chemin ?
2. Combien je peux perdre sur cette poche, dans quel scénario ?
3. Combien ça me coûte de sortir maintenant (slippage réel, gas) ?
4. Je sors tout, une partie, ou j'attends ? Et pourquoi ?
5. Combien de temps j'ai avant que la liquidité de sortie disparaisse ?

Ce qu'ils disent, ce que ça veut dire pour nous

Ce qu'ils disent	Ce qu'ils veulent vraiment	Implication pour notre projet
« Une position, une douzaine de dépendances silencieuses »	Voir les couches cachées (émetteur, hedges, custody, oracle)	Graphe profond (5+ couches), pas juste les contrats directs
« Le dur, c'est ce que ça fait à ton book »	Un montant, pas un feu rouge	Sortie en $ et en % de la poche, avec scénario
« Est-ce que le nombre est défendable »	Hypothèses visibles et sourcées	Chaque paramètre affiché avec sa source, analyse de sensibilité
« Stay or exit, and why »	Une action, à 3 h du matin	Verdict + seuil de probabilité + phrase d'explication
« Sur des données réelles »	Pas de mock, pas de slide	Adresses mainnet réelles, données live, replay d'un vrai incident

Douleurs typiques qu'on peut adresser : dépendances hors chaîne invisibles (Ethena couvre ses positions sur des exchanges de perp et confie ses actifs à des custodians) ; oracles qui ne suivent pas le marché (prix fixé ou taux de change interne : pas de liquidation, donc dette résiduelle chez les prêteurs) ; liquidité de sortie qui s'évapore pendant la crise ; même dépendance cachée dans plusieurs poches.

2. Define : le problème qu'on résout

Point de vue : un risk manager qui détient des positions DeFi composées a besoin de savoir en quelques minutes combien une défaillance n'importe où dans la pile lui coûte, comparé au coût certain de sortir, parce qu'une alerte seule ne lui dit pas quoi faire.

How Might We :

- Comment reconstruire automatiquement et de façon prouvée toute la pile d'une position à partir d'une seule adresse ?
- Comment traduire « le nœud X casse » en « la poche perd Y $ » avec des règles qu'un humain peut auditer ?
- Comment transformer l'incertitude sur la probabilité en une décision robuste, même si personne ne connaît la vraie probabilité ?
- Comment prouver au jury que ça marche sur une crise qui a vraiment eu lieu ?

Lecture fine des critères

Critère	Ce que le jury va regarder	Comment on prend les points	Piège à éviter
Découverte des dépendances (30 %)	Profondeur (5 couches comme l'exemple Pendle) et exactitude	Graphe construit par le code depuis l'adresse ; chaque arête marquée on-chain, API ou curée avec sa preuve	Graphe écrit à la main vendu comme automatique ; dépendances inventées par un LLM
Estimation d'impact (30 %)	Le nombre tient-il face à un expert ?	Formule simple, paramètres calibrés sur des incidents historiques, sensibilité affichée	Score magique 0-100, modèle boîte noire
Utilité de la décision (25 %)	Stay ou exit, et pourquoi	Perte attendue comparée au coût de sortie réel ; seuil de probabilité ; sortie partielle	Un niveau de risque sans action associée
Démo sur données réelles (15 %)	Ça tourne, sur du vrai	Adresses mainnet, appels live avec cache, replay d'un vrai incident	Démo qui plante ; données inventées

Conséquence : 60 % des points se jouent sur le graphe et le chiffre. L'UI sert à les montrer, pas à impressionner. On met l'effort là où sont les points.

3. Ideate : cinq concepts, un gagnant

Le concept 1 sort en tête (4,55/5) ; on lui greffe le 2 comme preuve et on garde le 3 en bonus. Notes de 1 à 5 par critère, total pondéré par les poids du jury.

Concept	Idée en une ligne	Découverte 30 %	Impact 30 %	Décision 25 %	Démo 15 %	Total
1. Rayon X de position	Adresse → graphe prouvé → choc sur un nœud → perte → STAY/EXIT	5	4	5	4	**4,55**
2. Replay d'incident	Rejouer une vraie crise et montrer que l'outil aurait vu le chemin	3	5	4	5	4,15
3. Carte de contagion du book	10 poches, dépendances partagées, concentration cachée	4	3	4	3	3,55
4. Audit qualité des oracles	Type d'oracle, heartbeat, déviation, source unique	3	3	2	4	2,90
5. Agent LLM qui lit les docs	Un agent découvre les dépendances en lisant la doc des protocoles	4	2	3	3	3,00

Pourquoi pas les autres seuls : le 4 est un sous-module du 1 (le type d'oracle est une arête du graphe), pas un projet. Le 5 est risqué sur l'exactitude, critère à 30 % : un LLM qui hallucine une dépendance détruit la crédibilité. On l'utilise seulement pour pré-remplir la couche curée hors chaîne, vérifiée par un humain, source à l'appui. Le 3 est séduisant (il colle à l'exemple des 10 poches du brief) mais il demande que le 1 marche d'abord.

Combinaison retenue : 1 = cœur du produit ; 2 = la preuve qu'on montre au jury ; 3 = extension si le cœur tourne de bout en bout au checkpoint de mi-parcours.

4. Le concept retenu : StackTrace

Une adresse en entrée, une fiche de risque actionnable en sortie : graphe prouvé, perte attendue en $, coût de sortie réel, verdict et seuil de bascule.

Une adresse en entrée, un verdict chiffré en sortie



pipeline StackTrace · 5 étapes

Chaque étape est le module d'une personne : Découverte et Graphe pour P1, Propagation et Décision pour P2, l'ensemble rendu visible par P3.

4.1 Positions de démo (Ethereum mainnet)

On choisit des positions qui ressemblent à ce que Belem fait vraiment (yield market-neutral) :

Position	Pourquoi elle est bonne pour la démo	Couches attendues
A. PT-sUSDe (Pendle) en collatéral sur Morpho, emprunt USDC	C'est littéralement l'exemple du brief	Ethereum → Ethena (sUSDe/USDe, hedges perp sur exchanges, custodians) → oracle du marché Morpho → AMM Pendle → marché Morpho + vault/curateur
B. Dépôt USDC dans un vault Morpho curé	Graphe large : le vault répartit sur N marchés, chacun avec son collatéral et son oracle	Vault → N marchés → N collatéraux → N oracles → émetteurs
C. (bonus) Vault qui déploie sur un venue de perp	Cas cité explicitement dans le brief	Vault → venue externe → bridge → contrepartie

Les marchés PT ont une échéance : on choisit au démarrage un marché encore actif et bien liquide.

4.2 Découverte des dépendances

Le graphe : nœuds typés (chaîne, actif, émetteur, oracle, AMM/venue de liquidité, marché de prêt, vault/curateur, contrepartie hors chaîne) ; arêtes typées (*est collatéral de*, *est pricé par*, *est rachetable en*, *est émis par*, *est déployé dans*, *est réglé sur*). Chaque arête porte un poids d'exposition, une **source** (on-chain, API, curée) et une **preuve** (adresse, appel, URL).

L'algorithme : un résolveur récursif avec un petit « adaptateur » par type de contrat. On part de l'adresse, on reconnaît le type, on lit ses composants, on recommence sur chaque composant jusqu'à une profondeur max.

- Pendle : le marché donne SY, PT, YT ; le SY donne le token sous-jacent.
- Morpho : l'id du marché donne token prêté, collatéral, oracle, modèle de taux, LLTV ; un vault donne sa liste de marchés et ses allocations.
- Oracles : on remonte jusqu'aux feeds (Chainlink, Redstone, Pyth) ou on détecte un prix fixé / taux interne. Ce détail est crucial pour l'impact.
- Tokens ERC-4626 : `asset()` donne le sous-jacent.
- Compléments par API : Pendle, Morpho (GraphQL), DeFiLlama (TVL, historique des hacks).
- Couche curée (fichier YAML, une source par ligne) pour ce qui n'est pas on-chain : custodians et exchanges d'Ethena, émetteur d'un stablecoin.

Les noms exacts des fonctions et les endpoints sont à vérifier en première heure ; jamais d'adresse recopiée de mémoire, toujours vérifiée sur Etherscan.

4.3 Estimation d'impact

Pour chaque nœud, quelques scénarios : exploit, depeg, panne ou manipulation d'oracle, gel de liquidité, arrêt de chaîne. Chaque scénario a une sévérité (perte de valeur du nœud) et une probabilité sur l'horizon choisi.

Propagation : une règle de transmission par type d'arête, écrite en clair. Exemples :

- Depeg de sUSDe de d % → le PT perd environ d % plus la décote de l'AMM.
- Collatéral qui chute, oracle qui suit le marché → liquidation, perte = bonus + slippage de liquidation.
- Collatéral qui chute, oracle fixé → pas de liquidation → dette résiduelle chez les prêteurs du vault. C'est le genre d'insight que le jury attend.

Calibration : probabilités de base par type de nœud tirées de l'historique des incidents (DeFiLlama hacks, rekt), ajustées par quelques facteurs simples (âge, TVL, type d'oracle, concentration). Sévérités calibrées sur des incidents réels. Monte Carlo sur les

défaillances (avec corrélation via les parents communs) → distribution de pertes : perte attendue, VaR 95 %, chemin le plus coûteux.

4.4 Règle de décision

```
\text{Exit si } \; p \cdot \mathrm{LGI} \cdot V \; > \; C_{exit}(V) \quad \Longleftrightarrow \quad p \; > \; p^{*} = \frac{C_{exit}(V)}{\mathrm{LGI} \cdot V}
```

p = probabilité que l'incident se matérialise sur l'horizon ; LGI = perte si incident (% de la poche) ; V = taille de la poche ; C_exit = slippage calculé sur la vraie liquidité + gas + rendement abandonné.

Exemple du brief : poche de 10 %, sortie à 11 % de slippage. Si l'incident coûterait 60 % de la poche, p* = 11 / 60 ≈ 18 %. Le manager n'a plus à estimer une perte, seulement à répondre : « est-ce que je crois à plus de 18 % de chances que ça tourne mal ? ». La décision reste robuste même si notre probabilité est contestée.

Sortie partielle : le slippage croit plus vite que la taille. On sort la fraction où le slippage marginal égale la perte attendue marginale : souvent la meilleure réponse n'est ni tout ni rien.

4.5 Ce que voit le manager : la fiche de risque

- Verdict : STAY / PARTIAL x % / EXIT, en une phrase.
- Perte attendue ($ et %), VaR 95 %, coût de sortie ($ et %), seuil p*.
- Les 3 chemins de contagion les plus coûteux, dessinés sur le graphe.
- La liste des hypothèses avec leur source, et un niveau de confiance.

5. Test : la démo et la validation

La démo tient en 3 minutes et suit les questions de Léa, dans l'ordre.

1. On colle l'adresse de la position A. Le graphe se construit couche par couche, chaque arête avec son badge on-chain / API / curée.
2. On clique sur le nœud USDe, scénario « depeg 30 % ». La perte se propage en couleur jusqu'à la position.
3. La fiche de risque s'affiche : perte attendue, coût de sortie calculé sur la vraie liquidité Pendle, p*, verdict PARTIAL ou EXIT, phrase d'explication.
4. On bascule sur le replay : un vault qui était exposé à la cascade Stream xUSD / Elixir deUSD de novembre 2025. L'outil retrouve le chemin de contagion depuis les données de l'époque.

5. Phrase de fin : « à 3 h du matin, Léa n'a qu'une question à se poser : est-ce que je crois à plus de 18 % de risque ? »

Le replay est à confirmer : il faut vérifier en début de hackathon qu'on retrouve les adresses et les allocations d'avant la crise (blocs historiques, API Morpho). Plan B : le depeg de USDe sur Binance du 10-11 octobre 2025, qui illustre parfaitement pourquoi la source de l'oracle compte.

Questions à poser aux mentors Belem (office hours, dès que possible)

- [ ] Quelles positions types ils trouveraient les plus parlantes : PT en collatéral, vault curé, vault vers perp ?
- [ ] Une couche curée hors chaîne (custodians, exchanges) est-elle acceptée si chaque ligne est sourcée ?
- [ ] Quel horizon de décision ils utilisent (24 h, 7 jours) et comment ils mesurent le coût de sortie aujourd'hui ?
- [ ] Le seuil p* leur parle-t-il plus qu'une probabilité estimée par nous ?
- [ ] Rester sur EVM seul est-il un handicap, ou Solana est-il un vrai plus ?

Critères pour valider l'idée entre vous trois

- [ ] Les trois comprennent la règle p* et savent l'expliquer en 30 secondes.
- [ ] On a vérifié qu'au moins un marché PT-sUSDe sur Morpho est actif aujourd'hui.
- [ ] On a un RPC Ethereum gratuit qui répond (Alchemy, Infura ou public).

6. Guideline d'équipe : comment on bosse

Trois modules indépendants reliés par un seul fichier JSON figé dès la 2e heure, et une règle d'or : `main` est toujours démontrable.

6.1 Rôles

Qui	Module	Livre	Points visés
Personne 1 « Découverte »	Adaptateurs on-chain + API, couche curée	`graph.json` réel pour les positions A et B	Découverte 30 %
Personne 2 « Moteur de risque »	Scénarios, propagation, Monte Carlo, coût de sortie, décision	`risk_report.json` à partir d'un graphe	Impact 30 % + Décision 25 %

Qui	Module	Livre	Points visés
Personne 3 « Démo et récit »	Interface, visualisation du graphe, fiche de risque, replay, pitch, README	L'app qui tourne + le pitch	Démo 15 % + tout le reste

P2 et P3 n'attendent pas P1 : ils démarrent sur un `graph.json` fait à la main, puis basculent sur le vrai.

6.2 Le contrat d'interface (à figer ensemble en 30 min)

```
{
  "position": {"id": "morpho:<marketId>:<user>", "value_usd": 1000000},
  "nodes": [
    {"id": "susde", "type": "asset", "label": "sUSDe", "address": "0x...",
"chain": "ethereum", "meta": {"tvl_usd": 0}}
  ],
  "edges": [
    {"from": "pt_susde", "to": "susde", "type": "redeemable_into", "weight":
1.0,
     "source": "onchain", "evidence": "SY.yieldToken() @ 0x..."}
  ]
}
```

Toute modification de ce format se décide à trois, jamais en solo.

6.3 Stack (simple, adaptée à des débutants)

- Python 3.11, `web3.py` et `requests` pour les données, `networkx` pour le graphe, `numpy` pour le Monte Carlo.
- Streamlit pour l'interface (zéro front-end à apprendre), `pyvis` ou `streamlit-agraph` pour le graphe.
- Un RPC Ethereum gratuit. **Tous les appels mis en cache sur disque** : la démo doit tourner même si le wifi ou le RPC lâche.

```
src/discovery/   adapters/ (pendle.py, morpho.py, oracles.py, erc4626.py),
curated.yaml, build_graph.py
src/risk/        scenarios.py, propagate.py, exit_cost.py, decide.py
app/             streamlit_app.py
data/            cache/, graphs/, replay/
docs/            CLAUDE.md, assumptions.md, pitch.md
```

6.4 Planning (en heures depuis le début du build)

1. **H0-H2 : validation.** Questions aux mentors, choix des marchés réels, contrat JSON figé, repo initialisé, `CLAUDE.md` écrit.
2. **H2-H5 : squelettes.** P1 lit un marché Morpho réel ; P2 calcule une perte sur le faux graphe ; P3 affiche le faux graphe.
3. **H5-H12 : cœur.** Graphe réel de la position A sur 5 couches ; propagation + p* ; fiche de risque.
4. **Checkpoint mi-parcours : bout en bout sur la position A.** Si ça ne marche pas, on coupe tout le reste et on répare.
5. **Mi-parcours → H-6 : profondeur.** Position B, calibration sourcée, sortie partielle, replay d'incident. Bonus : vue book 10 poches.
6. **H-6 : gel des fonctionnalités.** Plus de nouveau code, seulement des corrections.
7. **H-6 → fin : pitch.** README, `assumptions.md`, vidéo de secours de la démo, 3 répétitions chronométrées.

6.5 Maximiser les trois abonnements Claude Pro

- **Un `CLAUDE.md` à la racine du repo** : le brief, le contrat JSON, la stack, les conventions. Chaque session de Claude Code démarre alignée, peu importe le compte.
- **Chacun son module, chacun sa session.** Claude Code dans votre dossier ; vous ne travaillez jamais à deux sur le même fichier.
- **Planifier avant de coder :** demander d'abord un plan, le relire, puis faire coder par petites tâches testables. `/clear` entre deux tâches pour économiser le contexte.
- **Les limites Pro se rechargent par fenêtres de quelques heures** : décalez les gros chantiers entre vous pour qu'il y ait toujours un compte « frais ». Réservez le modèle le plus puissant au raisonnement (modèle de risque, formules, debug difficile), le modèle standard au code courant.
- **Ne jamais faire confiance à une adresse ou un chiffre venu de la mémoire d'un modèle** : vérifier sur Etherscan ou la doc officielle, et noter la source dans `assumptions.md`.
- **Utiliser Claude comme jury :** avant chaque checkpoint, demander à une session neuve de jouer le risk manager Belem et de démonter vos chiffres.

6.6 Règles git

- Une branche par personne (`discovery`, `risk`, `app`), fusion dans `main` toutes les 2 heures environ.
- Une seule personne fusionne (P3) ; on ne pousse jamais du code qui casse la démo.

- Messages de commit courts et en anglais ; `data/cache/` est commité pour que la démo tourne hors ligne.

7. Risques du projet et parade

Le plus gros risque pour des débutants n'est pas technique, c'est de vouloir tout faire : la parade est le checkpoint de mi-parcours.

Risque	Parade
Trop d'ambition, rien ne marche à la fin	Position A de bout en bout d'abord ; tout le reste est bonus
Le jury attaque nos probabilités	On montre p* : la décision ne dépend pas de notre probabilité, seulement de celle du manager ; sensibilité affichée
RPC limité ou wifi qui tombe pendant la démo	Cache disque commité + vidéo de secours
Marché PT expiré ou illiquide	Choisi et vérifié en H0
Données historiques du replay introuvables	Plan B : depeg USDe d'octobre 2025, ou replay simplifié sur l'état actuel
Conflits git, code écrasé	Un dossier par personne, une seule personne fusionne
Dépendance hallucinée par un LLM	Couche curée vérifiée par un humain, une source par ligne

Checklist avant de lancer le build

- [x] ~~Les trois ont lu ce doc et sont d'accord sur le concept StackTrace.~~
- [ ] Questions mentors posées (section 5), réponses notées ici.
- [ ] Rôles attribués nominativement.
- [ ] Marché réel de la position A identifié et vérifié sur Etherscan.
- [ ] Contrat `graph.json` figé et commité.
- [ ] `CLAUDE.md` écrit et commité.
- [ ] Heure du checkpoint de mi-parcours et du gel fixées selon la durée réelle du hackathon.